

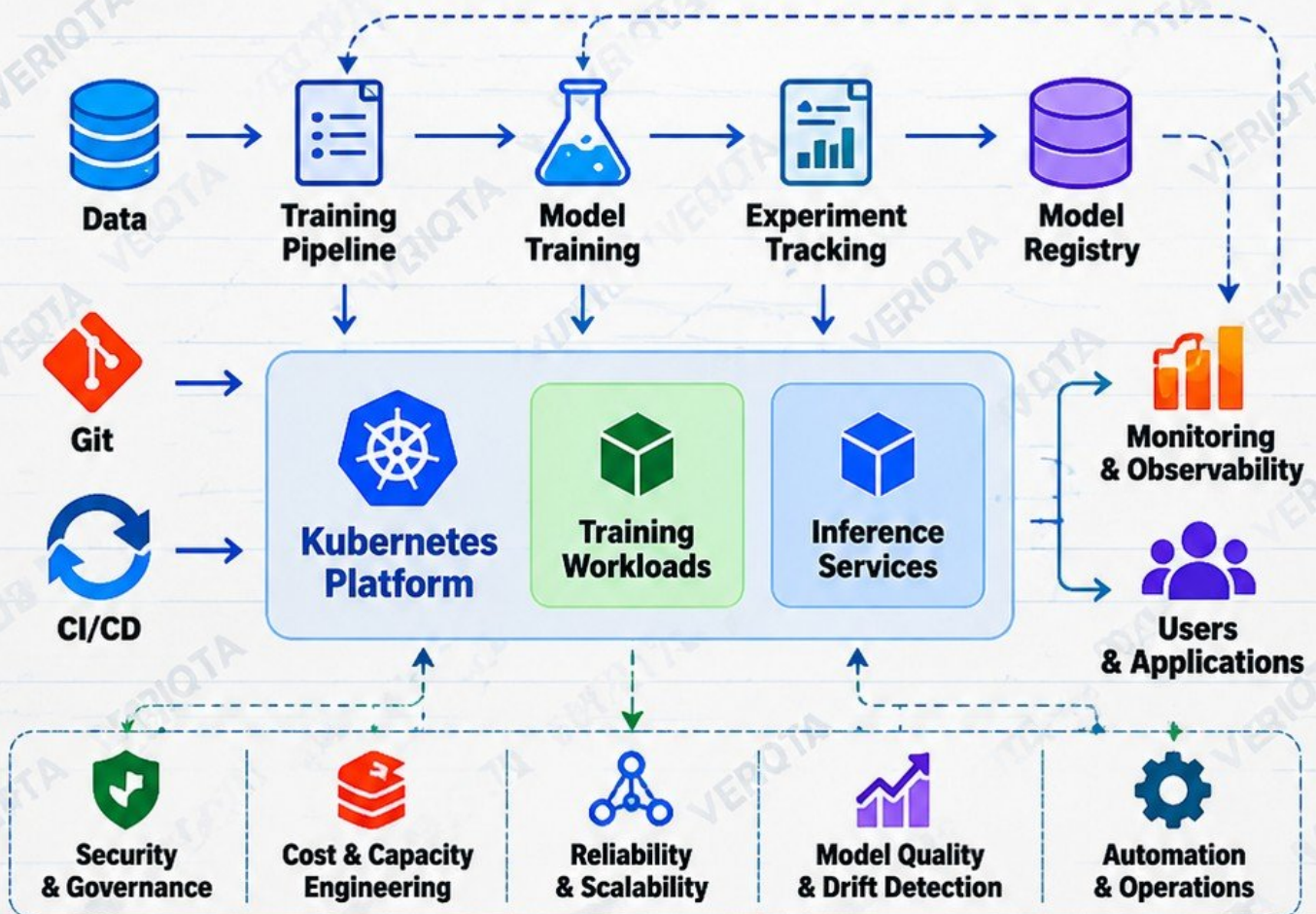
VERIQTA



MLOps

for DevOps & Platform Engineers

Design • Build • Deploy • Monitor • Operate
Machine Learning Systems at Scale



@veriqta



@veriqta



@veriqta



@veriqta



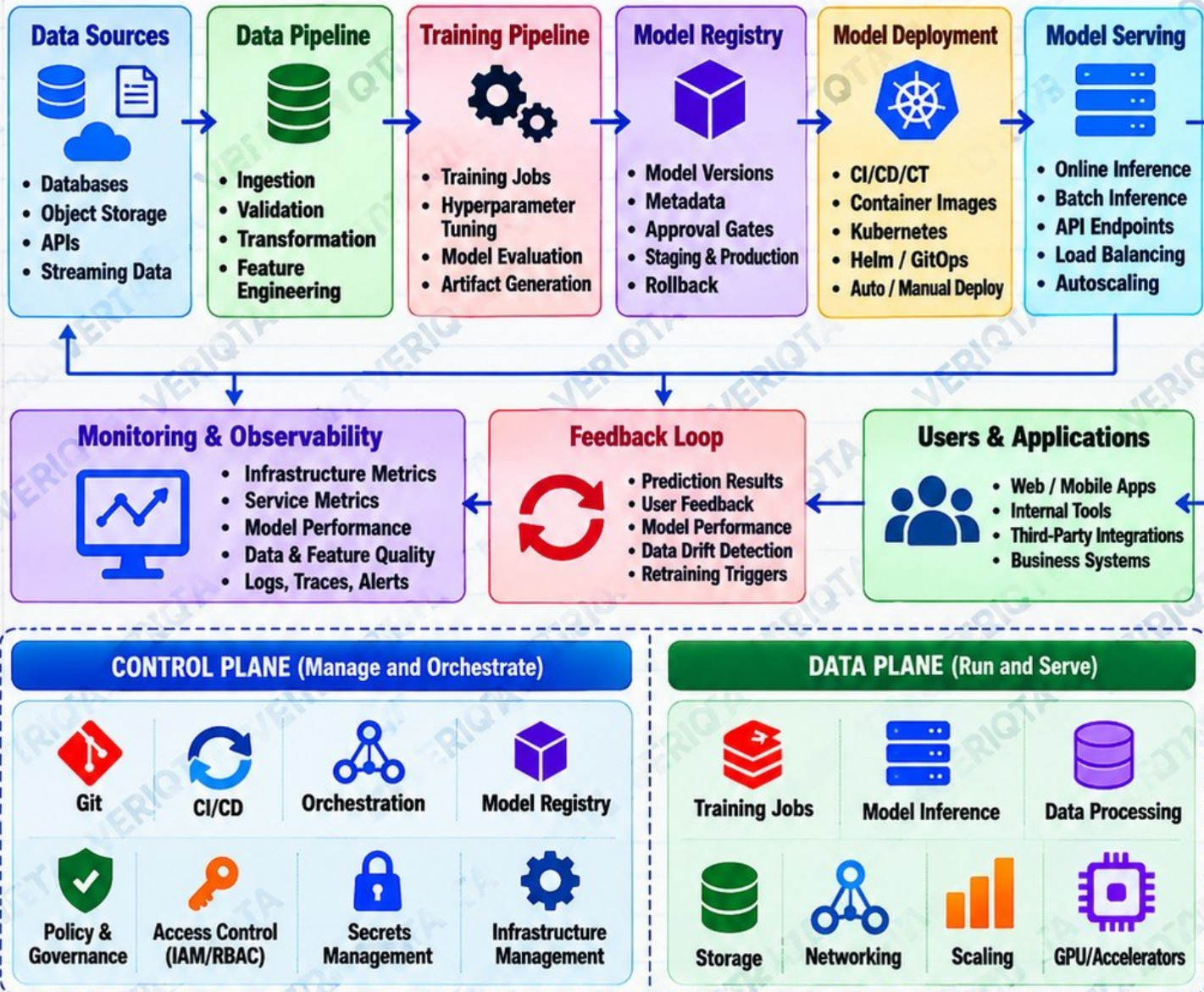
@veriqta



@veriqta

MLOps Production Architecture

How data, training, registry, deployment, inference, monitoring, and feedback form one production system. Control plane vs data plane and ownership boundaries.



KEY POINTS

- MLOps connects data, training, registry, deployment, inference, monitoring and feedback into one system.
- Control plane manages and orchestrates.
- Data plane runs training and serves models.
- Clear ownership boundaries reduce risk and improve operability.
- Feedback from real usage drives model improvement.

PRODUCTION CONSIDERATIONS

- Reproducible pipelines and artifacts.
- Version everything: code, data, model, configuration.
- Apply security, access control and audit logging.
- Monitor both system health and model quality.
- Plan for failure, rollback and retraining.
- Design for scale and cost efficiency.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



ML Lifecycle vs DevOps Lifecycle

Why shipping a model differs from shipping ordinary application code.

Code + data + model + configuration + environment must be versioned and reproducible.

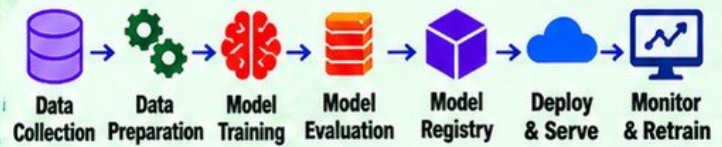
Traditional DevOps Lifecycle (Application Code)



Key Characteristics

- Primarily code and configuration.
- Deterministic builds.
- Same artifact deployed across environments.
- Easier to reproduce and roll back.
- Changes are mostly code, dependencies and configs.

ML Lifecycle (Machine Learning Systems)



Key Characteristics

- Involves code, data, model artifacts, configuration and environment.
- Data and model changes affect behaviour.
- Training is often non-deterministic.
- Models can drift over time.
- Reproducibility requires versioning data, code, environment and model artifacts.
- Deployment needs additional validation and monitoring.

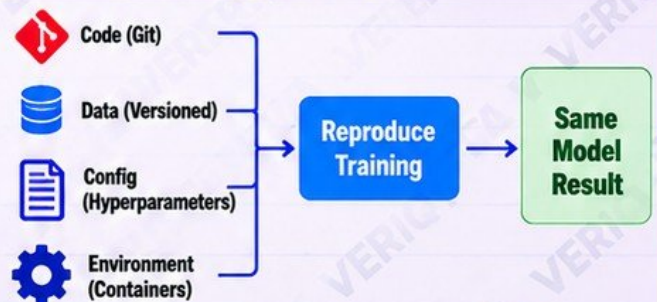
What Makes ML Different?

- 1 Data is part of the product – training data, validation data and feature data must be versioned.
- 2 Model artifacts are large and environment-dependent – GPU, drivers, libraries and framework versions matter.
- 3 Model behaviour can change over time – even without code changes due to data drift.
- 4 Reproducibility is harder – same code and data can produce slightly different models.
- 5 Additional validation is required – model performance, bias, drift and real-world behaviour.
- 6 Lifecycle continues after deployment – monitoring, feedback and retraining are ongoing.

Versioning in ML Systems

Component	What to Version
Code	Source code, training scripts, inference code
Data	Datasets, feature sets, data versions, schemas
Configuration	Hyperparameters, environment configs, pipeline configs
Model	Model artifacts, versions, metadata
Environment	Dependencies, container images, libraries, GPU/CPU versions

Reproducibility Flow



Real World Example

- A model trained on last month's data is deployed.
- Next month, data distribution changes.
- Model performance drops even though code did not change.
- This is why monitoring, data versioning and retraining are essential.

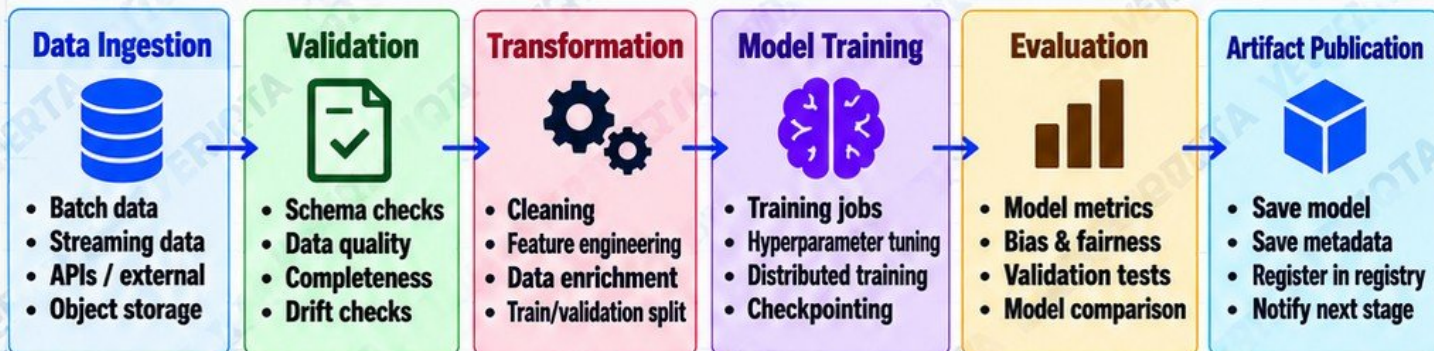


Senior Engineer Takeaway

- Treat ML as a complete system, not just code.
- Version everything: code, data, model, config and environment.
- Ensure reproducibility across environments.
- Build pipelines and monitoring for continuous validation.
- Plan for drift, retraining and lifecycle management.

Training Pipeline Architecture

Ingestion → validation → transformation → training → evaluation → artifact publication.
Idempotency, retries, orchestration, failure recovery, and expensive reruns.



Orchestration Layer (Manage, Schedule and Recover)



Key Design Principles

- 1 Idempotent steps – safe to rerun without duplicates.
- 2 Retries for transient failures (network, storage, API).
- 3 Clear input and output contracts between stages.
- 4 Version data, code, configuration and environment.
- 5 Save intermediate artifacts and metadata.
- 6 Support partial reruns to avoid full pipeline cost.
- 7 Use orchestration (e.g. Airflow, Argo, Kubeflow) for scheduling, dependencies and monitoring.
- 8 Design for failure recovery and alerting.

Common Failure Points

- 1 Data ingestion failures (source unavailable).
- 2 Data quality issues (missing, corrupt or schema changes).
- 3 Transformation errors (invalid data, feature bugs).
- 4 Training job failures (out of memory, GPU issues, timeouts).
- 5 Evaluation failures (invalid metrics, data leakage).
- 6 Artifact publication failures (registry, storage, permission).
- 7 Orchestrator failures (stuck jobs, missed dependencies).
- 8 High cost due to repeated full training runs.
- 9 Inconsistent environments causing non-reproducible results.

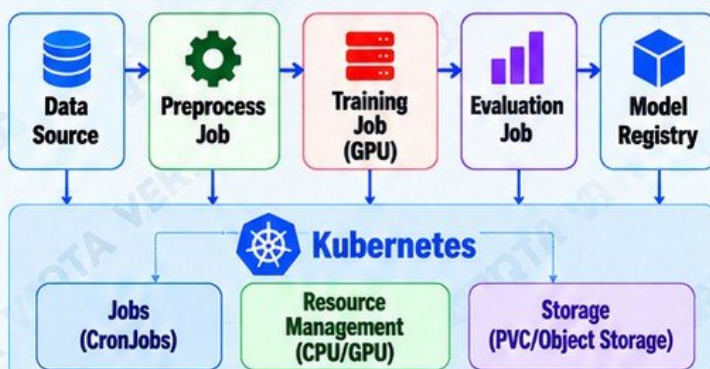
Example Orchestration Tools



Real World Considerations

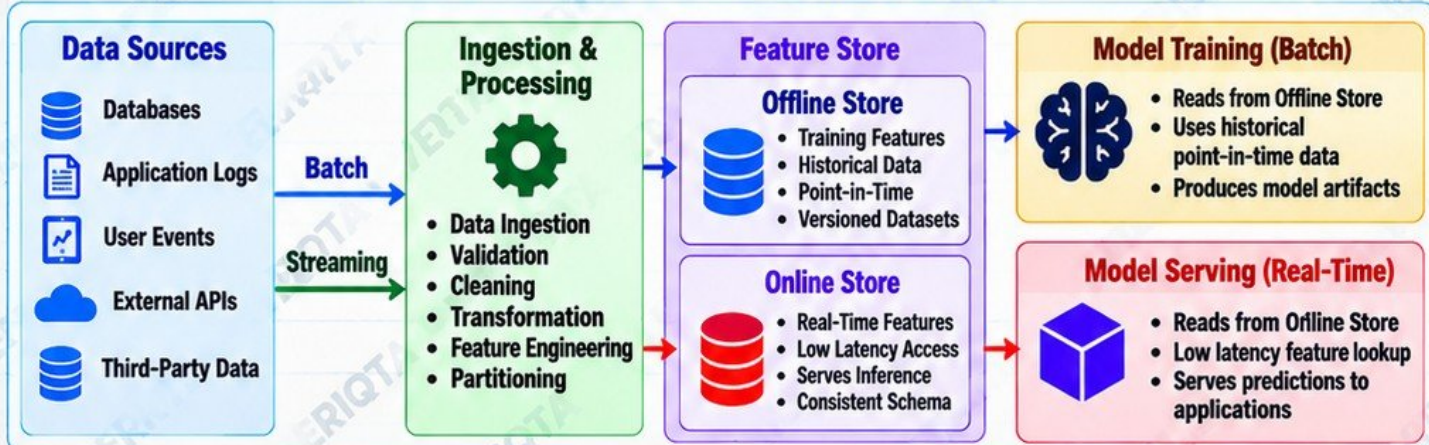
- Training can be expensive – use caching and partial reruns.
- Ensure data and environment versioning for reproducibility.
- Use separate compute for training and other workloads.
- Plan for long running jobs and spot/preemptible instances.
- Monitor pipeline health and set alerts for failures and delays.

Simplified Training Pipeline on Kubernetes



Data & Feature Pipelines

Batch vs streaming features, offline/online stores, point-in-time correctness, lineage, freshness, schema changes, and training-serving skew.



Batch vs Streaming Features

Batch Features



- Processed in batches
- Used for training
- Higher latency
- Large historical data
- Cost efficient

Streaming Features



- Processed in real time
- Used for online inference
- Low latency
- Recent and continuous data
- Higher operational complexity

Point-in-Time Correctness



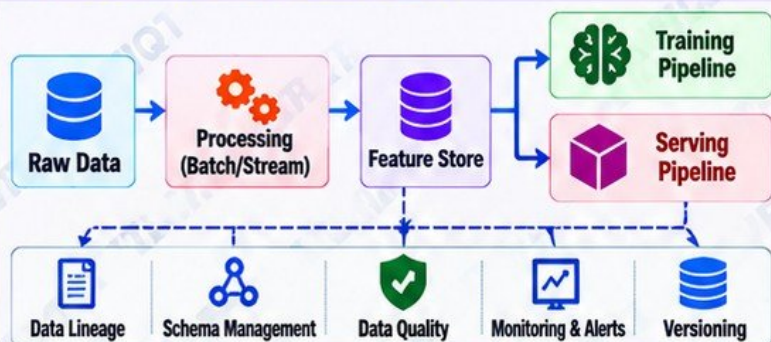
- Use only data available at the time of prediction or training.
- Prevent future data leakage.
- Ensure correct joins and time windows.
- Critical for accurate model evaluation.

Training-Serving Skew



- Occurs when training and serving use different data, features or logic.
- Leads to poor model performance in production.
- Keep feature definitions, transformations and schemas consistent.

Feature Pipeline Architecture



Key Considerations

- 1 Freshness – how up to date are the features?
- 2 Schema changes – handle without breaking pipelines.
- 3 Lineage – track data from source to feature.
- 4 Consistency – same logic for training and serving.
- 5 Idempotency – safe to rerun without duplicates.
- 6 Scalability – handle data and feature growth.
- 7 Fault tolerance – retries and failure recovery.
- 8 Cost – storage, processing and serving costs.
- 9 Data governance – access control and auditing.



Real World Example

- User activity events stream into a feature pipeline.
- Features are written to an online store for real-time inference.
- The same feature logic and historical data are used for training.
- Point-in-time windows prevent data leakage.
- Monitoring alerts if feature freshness or schema changes.



Common Failure Points

- Incorrect feature joins (future data leakage).
- Schema changes breaking pipelines.
- Stale or missing features in online store.
- Inconsistent feature logic between training and serving.
- Pipeline failures due to data quality issues.
- High latency in feature serving.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



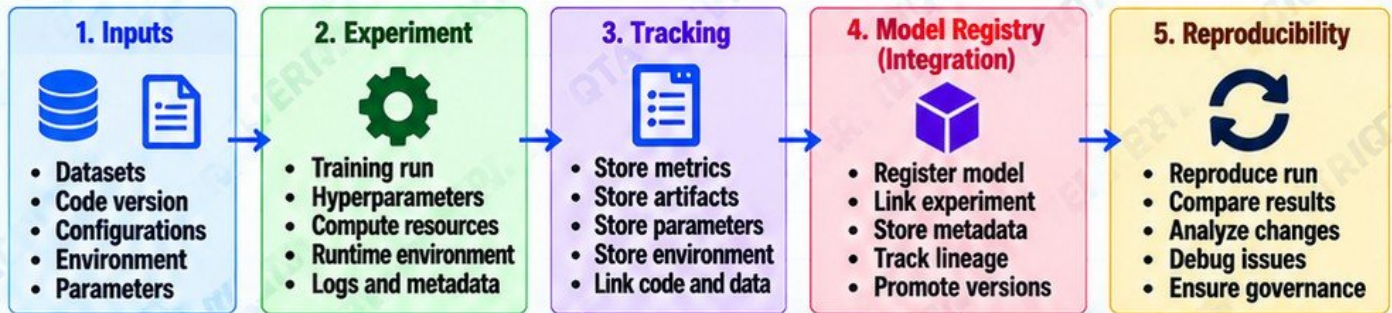
@veriqta



Experiment Tracking

Tracking parameters, metrics, datasets, code revisions, environments, artifacts, and reproducibility. Why “which model produced this result?” must always be answerable.

Experiment Tracking Flow



What to Track

- Parameters** – hyperparameters, feature settings
- Metrics** – training and validation metrics
- Datasets** – name, version, schema, source
- Code** – git commit, branch, repository
- Environment** – libraries, framework versions
- Artifacts** – model files, logs, plots, checkpoints
- Compute** – instance type, GPU/CPU, runtime
- Metadata** – tags, notes, purpose, owner

Typical Experiment Tracking Tools

- | MLflow | Weights & Biases | Comet | Neptune |
|--|---|---|--|
| <ul style="list-style-type: none"> Open source Metrics & artifacts Model registry Wide ecosystem | <ul style="list-style-type: none"> Visual tracking Collaboration Metrics & artifacts Model versioning | <ul style="list-style-type: none"> Experiment tracking Model evaluation Dataset versioning Team collaboration | <ul style="list-style-type: none"> Metadata tracking Model governance Reproducibility Enterprise ready |

Why Experiment Tracking Matters

- Know exactly which model produced a result.
- Compare experiments fairly.
- Reproduce training runs.
- Understand the impact of data and parameter changes.
- Debug issues and performance drops.
- Meet governance and audit requirements.
- Enable safe model promotion and rollback.

Example Experiment Record

Field	Example Value
Experiment Name	churn-prediction-v3
Git Commit	a3f9c2d
Dataset Version	customer-data-v5
Parameters	lr=0.001, batch=64, epochs=50
Metrics	accuracy=0.92, f1=0.89
Environment	Python 3.10, PyTorch 2.1, CUDA 12.1
Compute	g5.2xlarge (1 x A10G)
Artifacts	model.pt, training.log, confusion_matrix.png
Registered Model	churn-model:1.3
Status	Completed

Model Lineage Example



Best Practices

- Version everything – code, data, config and environment.
- Use consistent naming and tagging.
- Store all artifacts and metadata.
- Make experiments reproducible.
- Integrate with model registry and CI/CD.

Common Failure Points

- Missing dataset or data version information.
- Cannot reproduce a training run.
- Inconsistent environment or library versions.
- Metrics not tracked or tracked incorrectly.
- No link between experiment and registered model.
- Lost artifacts or incomplete metadata.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Model Registry & Promotion

Model versions, metadata, approval gates, staging, production promotion, rollback, lineage, governance, and artifact immutability.

Model Registry & Promotion Flow

1. Trained Model



- Model artifact
- Metrics
- Parameters
- Environment
- Training code

2. Register



- Create version
- Store metadata
- Link to dataset
- Link to code
- Mark as immutable

3. Approval Gates



- Validation checks
- Quality metrics
- Security scanning
- Business approval
- Governance policies

4. Staging



- Deploy to staging
- Run integration tests
- Shadow or canary
- Validate performance
- Validate data behavior

5. Production



- Promote approved model
- Serve live traffic
- Monitor metrics
- Track version usage
- Enable rollback

Model Registry Contents

- Model Versions – unique version for each trained model
- Metadata – parameters, metrics, datasets, code, environment
- Artifacts – model files, logs, evaluation results
- Code Reference – git commit, branch, repository
- Dataset Reference – dataset version and schema
- Environment – libraries, framework, hardware (GPU/CPU)
- Tags – model type, use case, owner, stage
- Lineage – links to training data, code, experiments
- Governance Info – approvals, policies, audit logs

Typical Model Registry Tools



MLflow

- Model registry
- Experiment tracking
- Metadata
- Stage management



SageMaker

- Model registry
- Versioning
- Approval workflows
- CI/CD integration



Vertex AI

- Model registry
- Metadata & lineage
- Governance
- Production deployment



Kubeflow

- Model registry
- Pipeline integration
- Multi-cloud
- Open source

Promotion Stages

Development

Initial model registration and testing

Staging

Validation, integration tests, shadow or canary deployment

Production

Live traffic monitoring and continuous evaluation

Example Model Registry Entry

Field	Example Value
Model Name	fraud-detection-model
Version	v2.3.1
Description	Detects fraudulent transactions
Parameters	max_depth=8, learning_rate=0.01
Metrics	accuracy=0.96, precision=0.94, recall=0.92
Dataset Version	transactions-dataset-v5
Code Commit	a3f9c2d (main branch)
Environment	Python 3.10, PyTorch 2.1, CUDA 12.1
Stage	Staging
Status	Approved
Registered At	2026-10-01 14:30:22
Owner	ml-platform-team

Rollback Strategy



- Keep previous model versions available.
- Use blue green or canary deployment.
- Automate rollback based on metrics or alerts.
- Validate rollback with smoke tests.
- Ensure data and schema compatibility.
- Document and audit rollback events.



Key Governance Considerations

- Only approved models can be promoted to production.
- Track lineage from data to model to deployment.
- Enforce access control and audit logging.
- Ensure artifact immutability (no changes after registration).
- Monitor model usage and performance.
- Comply with organisational and regulatory requirements.



Best Practices

- Register every trained model with complete metadata.
- Use consistent naming and versioning.
- Automate validation and approval gates.
- Keep model artifacts immutable.
- Track lineage for reproducibility and audits.
- Use staging with realistic data before production.
- Monitor after promotion.
- Plan and test rollback.
- Integrate with CI/CD and infrastructure as code.



Common Failure Points

- Missing or incomplete metadata.
- Inconsistent versioning and naming.
- Skipping validation before promotion.
- No rollback plan.
- Unauthorized model changes in production.
- Lack of lineage and audit logs.
- Serving a model with incompatible data or schema.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



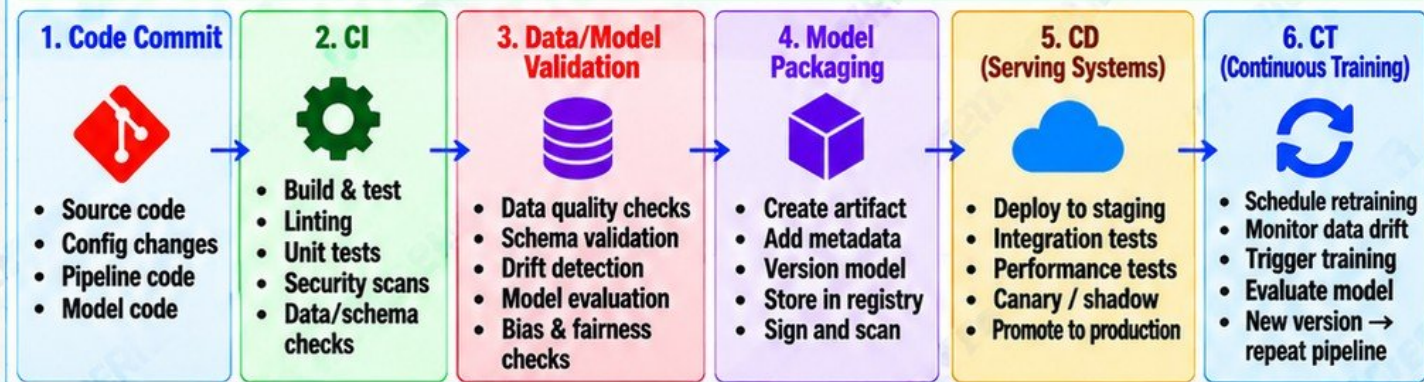
@veriqta



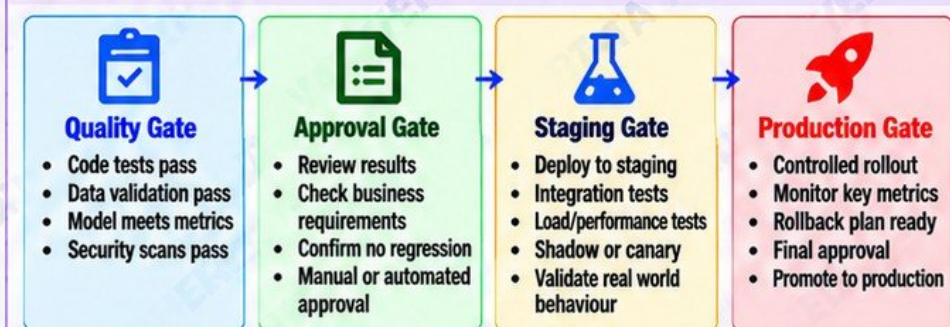
CI/CD/CT for ML

CI for code, data/model validation, CD for serving systems, and continuous training. Promotion gates and why retraining should not automatically mean deployment.

End-to-End CI/CD/CT Pipeline for ML



Promotion Gates

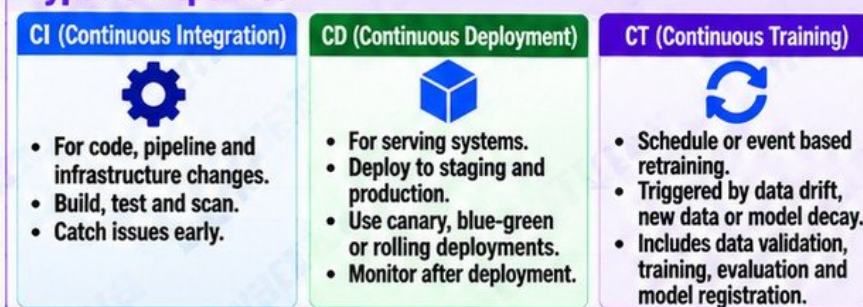


Why Retraining ≠ Deployment

- New model may not be better.
- Can perform worse on real data.
- May increase latency or cost.
- Can have bias, drift or data issues.
- Needs validation, testing and approval.
- Automatic deployment can cause outages, bad predictions or business risk.

Retraining should create a new model candidate. Deployment should only happen after proper validation and approval.

Types of Pipelines



Typical Tools



Real World Example Flow



Best Practices

- Separate training, validation and deployment.
- Use consistent versioning and metadata.
- Enforce quality and approval gates.
- Test in staging with real world data.
- Use canary or shadow deployments.
- Monitor model performance after deployment.
- Automate where possible, but keep human oversight.
- Track lineage from data to deployed model.
- Have a rollback plan.

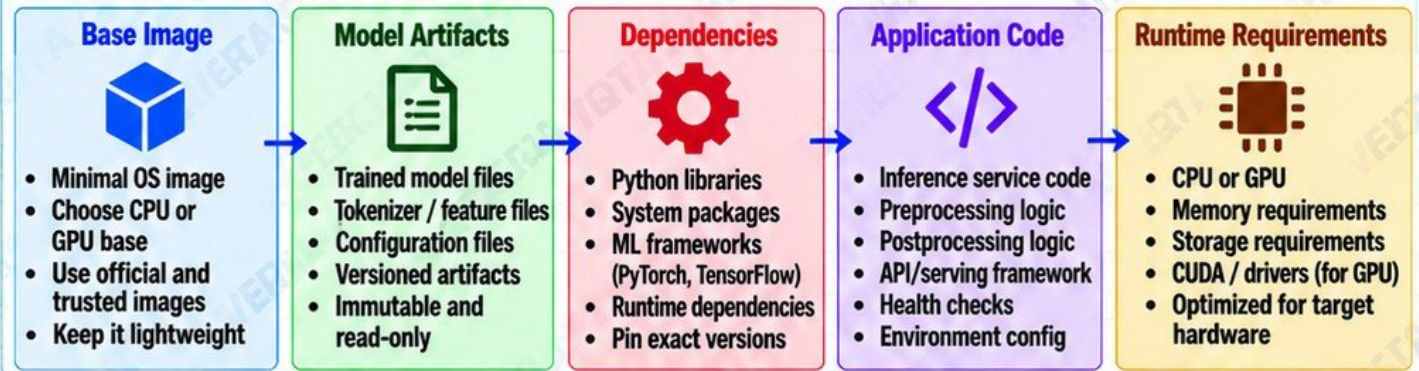
Common Failure Points

- Skipping data validation.
- Deploying without proper testing.
- Automatically deploying every retrained model.
- No rollback plan.
- Inconsistent environments.
- Poor monitoring after deployment.

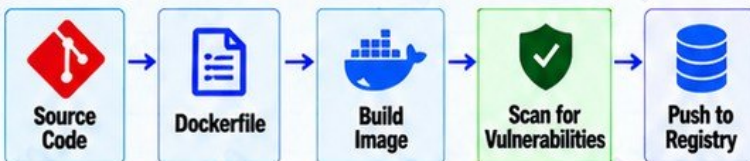
Packaging ML Workloads

Containers, model artifacts, dependencies, CPU/GPU requirements, image size, startup behavior, immutable artifacts, vulnerability scanning, and reproducible environments.

What Goes Into an ML Container?



Container Build Process



Example Dockerfile (Simplified)

```

FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY model/ ./model/
COPY app/ ./app/
ENV MODEL_PATH=/app/model
EXPOSE 8000
CMD ["python", "app/server.py"]
  
```

CPU vs GPU Images

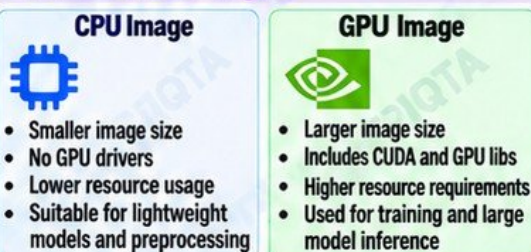


Image Size Considerations

- Use minimal base images.
- Remove unnecessary packages.
- Use multi-stage builds.
- Compress model artifacts.
- Avoid copying large datasets into the image.
- Keep only runtime files.

Startup Behavior

- Fast startup for inference services.
- Lazy load large models when possible.
- Warm up models after startup.
- Use health checks and readiness probes.
- Avoid long initialization steps.

Immutable Artifacts

- Model artifacts should not change once built.
- Use versioned file names or digests.
- Store in artifact registry.
- Do not modify models inside the container.
- Rebuild image for any changes.

Vulnerability Scanning

- Scan base images and dependencies.
- Check for known CVEs.
- Use automated scanning in CI.
- Keep runtime libraries updated.
- Track and fix high and critical issues.

Reproducible Environments

- Pin library and system package versions.
- Use lock files (requirements.txt, poetry, conda).
- Track model artifact versions.
- Use the same environment for build, test and production.
- Document the environment and image digest.



Best Practices

- Use minimal and trusted base images.
- Pin all dependency versions.
- Keep images small and secure.
- Use multi-stage builds.
- Separate build and runtime dependencies.
- Store models in an artifact registry.
- Scan images for vulnerabilities.
- Make artifacts immutable.
- Test container startup and health checks.
- Ensure the environment is reproducible.



Common Failure Points

- Large image size and slow deployment.
- Missing GPU drivers or incompatible CUDA version.
- Dependency conflicts and version mismatches.
- Security vulnerabilities in base image or libraries.
- Long startup time and model loading failures.
- Non-reproducible environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Kubernetes for ML Workloads

Training jobs vs inference services, scheduling, Jobs, resource requests, accelerators, storage, namespaces, quotas, workload isolation, and failure domains.

Kubernetes for ML Overview



- Run training and inference workloads at scale
- Efficiently use CPU, GPU and storage
- Provide isolation, reliability and scalability
- Manage the full ML lifecycle in production

Training Jobs vs Inference Services

Training Jobs (batch workload)



- Run as Jobs or scheduled Jobs
- Typically long running
- Use high CPU/GPU and large storage
- Process large datasets
- Retry on failure
- Do not require low latency

Inference Services (real-time workload)



- Run as Deployments or StatefulSets
- Serve predictions via API
- Require low latency and high availability
- Scale based on traffic
- Use GPUs or CPUs
- Need health checks and auto scaling

Key Kubernetes Resources



Pods (run containers)



Deployments (inference)



Jobs (training)



CronJobs (scheduled)



PersistentVolumes (storage)



Services (expose apps)



Ingress (external access)



Namespaces (isolation)



ResourceQuotas (limit usage)

ML Workload Architecture on Kubernetes

Training Workflow



Inference Workflow



Resource Requests and Limits



CPU

- Set requests and limits
- Prevent resource starvation



GPU

- Use NVIDIA GPU resources
- Request specific GPU count (e.g. nvidia.com/gpu)



Memory

- Set memory requests and limits
- Avoid out of memory (OOM)



Storage

- Use PersistentVolumes (PV)
- Choose fast storage for training
- Use ReadWriteMany if needed

Scheduling and Accelerators



Scheduling

- Use node selectors and taints
- Schedule GPU nodes for training
- Spread workloads across nodes
- Use topology awareness



Accelerators

- Use GPU nodes for training and inference
- Support for NVIDIA GPUs
- Use device plugins
- Consider multi-GPU training

Namespaces and Workload Isolation



Namespaces

- Separate dev, staging and production
- Isolate teams and workloads
- Use ResourceQuotas
- Apply network policies
- Prevent noisy neighbour issues

Failure Domains and Reliability



- Distribute workloads across nodes and zones
- Use replication for inference services
- Handle node failures with rescheduling
- Use Pod disruption budgets (PDB)
- Consider multi-zone deployments
- Design for graceful degradation

Storage for ML Workloads



- Use PersistentVolumes (PV) and PersistentVolumeClaims (PVC)
- Use fast storage for training datasets
- Use shared storage for distributed training
- Choose suitable storage class (e.g. SSD, object storage)

Networking and Access



- Use Services for internal access
- Use Ingress for external access
- Implement authentication and TLS
- Control access with network policies
- Isolate inference endpoints

Best Practices



- Use resource requests and limits
- Separate training and inference workloads
- Use namespaces and quotas
- Monitor GPU and storage usage
- Implement auto scaling for inference
- Design for failure and recovery



Common Failure Points

- Insufficient CPU/GPU resources
- Pods stuck in pending state
- Out of memory (OOM) errors
- Slow or failed data access
- Misconfigured storage or permissions
- Noisy neighbour affecting performance



Production Considerations

- Monitoring and alerting
- Logging and debugging
- Security and compliance
- Cost optimization



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



GPU Scheduling & Isolation

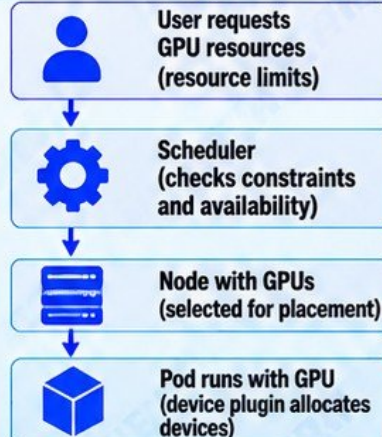
GPU requests, device plugins, scheduling constraints, utilization, fragmentation, expensive idle capacity, workload placement, and GPU-aware platform decisions.

Why GPU Scheduling Matters



- GPUs are expensive resources.
- Demand is high and supply is limited.
- Poor scheduling leads to idle capacity.
- Correct scheduling improves utilization.
- Isolation prevents noisy neighbours.
- Proper placement reduces cost and latency.

GPU Scheduling Flow



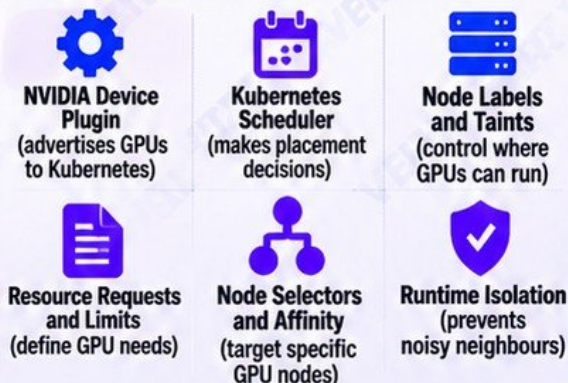
GPU Resource Requests in Kubernetes

```

apiVersion: v1
kind: Pod
metadata:
  name: gpu-training
spec:
  containers:
    - name: trainer
      image: pytorch/pytorch:2.1
      resources:
        requests:
          nvidia.com/gpu: 1
        limits:
          nvidia.com/gpu: 1
  
```



Key Components



Scheduling Constraints



GPU Utilization and Monitoring



- Track GPU utilization (e.g. nvidia-smi).
- Monitor memory usage, temperature and power.
- Identify idle GPUs.
- Use metrics and alerts.
- Detect underutilized or overprovisioned nodes.
- Optimize workload placement based on utilization.

Workload Placement Strategies



Fragmentation Issues



- GPUs can be underutilized due to uneven requests.
- Small workloads may not fit on large GPUs.
- Leads to wasted capacity and higher cost.
- Use right sizing and batching where possible.
- Consider GPU sharing technologies (e.g. MIG) if supported.

Isolation Techniques



- Use dedicated GPU nodes for sensitive workloads.
- Apply Kubernetes resource quotas.
- Use namespaces for team isolation.
- Prevent noisy neighbours from impacting performance.
- Consider GPU partitioning (e.g. MIG) for multi-tenancy.

GPU-aware Platform Decisions



- Choose the right GPU instance types.
- Balance cost, performance and availability.
- Plan for autoscaling of GPU workloads.
- Use spot instances where appropriate.
- Design for failure and recovery.
- Align platform design with ML workload needs.



Best Practices

- Request the right amount of GPU resources.
- Use node selectors, affinity and taints.
- Monitor utilization and cost.
- Avoid long running idle workloads.
- Use namespaces and quotas for isolation.
- Plan for scaling and failure scenarios.



Common Failure Points

- Pods stuck in Pending due to no available GPUs.
- Incorrect resource requests or limits.
- GPU driver or device plugin issues.
- Fragmented GPU capacity.
- Noisy neighbour workloads.
- High cost due to idle or overprovisioned GPUs.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Model Serving Architecture

Online, batch, asynchronous and streaming inference. Request path, preprocessing, model runtime, post-processing, dependencies, latency budgets, and failure behavior.

Model Serving Types

Online Inference (Real-time)



- Low latency
- Synchronous request and response
- Used for live applications (e.g. chat, recommendations)

Batch Inference



- Process large datasets
- Higher latency is OK
- Common for reports, analytics and offline predictions

Asynchronous Inference



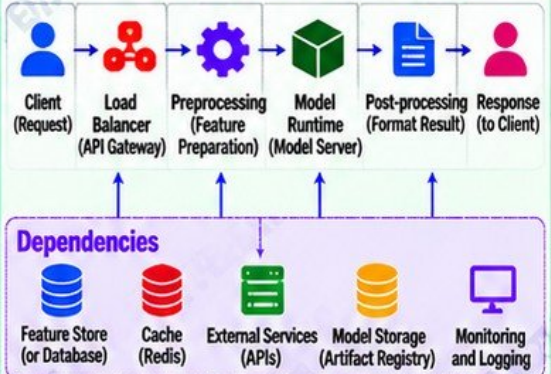
- Request is accepted and processed later
- Uses queues or event driven systems
- Good for long running inference tasks

Streaming Inference

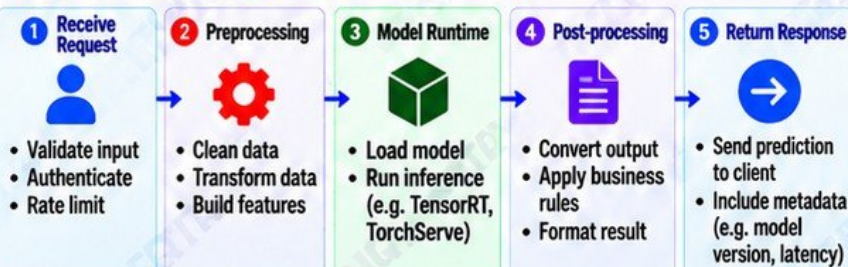


- Real-time continuous data processing
- Used for time series, sensor and event data
- Requires state management and low latency processing

High Level Model Serving Architecture



Request Path Flow (Online Inference)



Typical Model Serving Tools



Inference Patterns

Synchronous (Online)



- Immediate response
- Low latency
- User facing apps

Asynchronous



- Request queued
- Processed later
- Used for long running tasks

Batch



- Process many records
- Higher latency
- Offline predictions

Streaming



- Continuous data flow
- Real-time processing
- Stateful inference

Latency Budget Breakdown

Component	Typical Time (ms)
Network / Load Balancer	5 - 20
Authentication & Validation	5 - 10
Preprocessing / Feature Engineering	10 - 50
Model Inference (runtime)	10 - 200+
Post-processing	5 - 20
Total (example)	50 - 300+

Failure Behavior



- Handle model server failures (retries, fallback).
- Use timeouts and circuit breakers.
- Return meaningful error messages.
- Use fallback models or cached results where possible.
- Monitor and alert on high error rates and latency.

Scalability Considerations



- Use horizontal scaling.
- Scale based on CPU, GPU and request latency.
- Use auto scaling and queue based systems.
- Consider model sharding for large models.
- Plan for multi region deployment if required.

Key Considerations



- Choose the right serving pattern for the use case.
- Manage dependencies (feature store, cache, databases, external APIs).
- Meet latency and throughput requirements.
- Ensure observability (metrics, logs, tracing).
- Plan for failure, scaling and cost.
- Keep model and environment versions consistent.



Common Issues

- High latency due to preprocessing or external calls.
- Model server overload.
- Dependency failures (database, feature store, API).
- Inconsistent input data and schema.
- Memory leaks and resource exhaustion.
- No fallback strategy.



Best Practices

- Keep models and preprocessing code versioned.
- Use efficient model runtimes (TensorRT, TorchServe, Triton).
- Monitor latency, error rate and resource usage.
- Use caching for repeated or expensive requests.
- Implement retries, timeouts and circuit breakers.
- Test failure scenarios and load test before production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Inference Scaling

Horizontal scaling, concurrency, batching, queues, CPU/GPU saturation, cold starts, model loading, autoscaling signals, backpressure, and cost/latency trade-offs.

Why Inference Scaling Matters

- Handle increasing request traffic.
- Maintain low latency and high availability.
- Efficiently use CPU and GPU resources.
- Avoid overload and service failures.
- Optimize cost while meeting performance targets.



Inference Scaling Challenges



- Sudden traffic spikes.
- Model loading time and cold starts.
- GPU or CPU saturation.
- Managing concurrent requests.
- Keeping latency within limits.
- Balancing cost and performance.
- Preventing queue build-up and request drops.

Key Scaling Metrics



Latency
(p50, p95, p99)



Throughput
(requests per second)

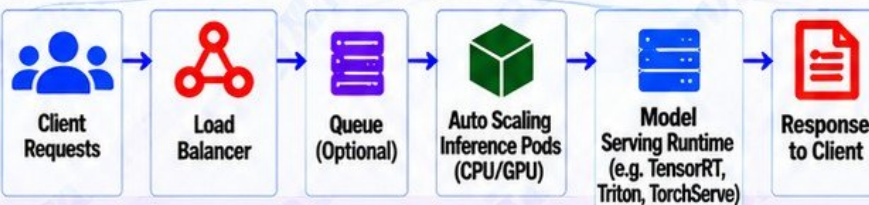


CPU/GPU
Utilization

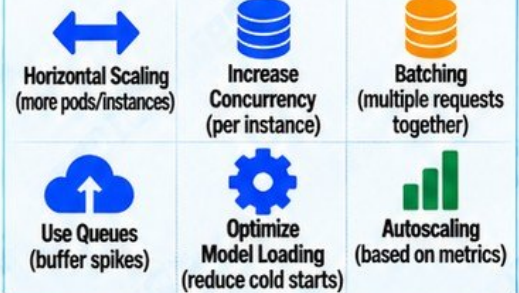


Concurrency
(active requests)

Inference Scaling Architecture



Scaling Strategies



Concurrency and Batching

Concurrency



- Handle multiple requests at the same time.
- Useful for I/O bound or lightweight models.
- Limited by CPU/GPU resources and memory.

Batching



- Combine multiple requests into one batch.
- Improves GPU utilization and throughput.
- Increases latency slightly due to batching delay.
- Best for high traffic scenarios.

CPU vs GPU Saturation

CPU Saturation



- High CPU usage.
- Request processing slows down.
- Increase instances or optimize code.
- Consider moving to GPU for heavy models.

GPU Saturation



- High GPU utilization (close to 100%).
- Requests queue up.
- Increase GPU instances or use batching.
- Monitor memory usage to avoid OOM errors.

Cold Starts and Model Loading

Cold Starts



- Delay when a new instance starts.
- Model weights need to be loaded into memory.
- More common in serverless or autoscaled environments.
- Can cause high initial latency.



Reduce Cold Start Impact

- Use pre-warmed instances.
- Keep models in memory.
- Use smaller model sizes or optimized formats (e.g. ONNX, TensorRT).
- Use model caching and faster storage.

Autoscaling Signals

- CPU or GPU utilization thresholds.
- Request rate (RPS).
- Queue length.
- Latency (e.g. p95).
- Custom application metrics (e.g. model queue time).
- Scale out when thresholds are high.
- Scale in when demand decreases.



Backpressure and Queues

- Use queues to buffer incoming requests during spikes.
- Prevent system overload.
- Apply backpressure when queues are too long (reject or slow requests).
- Choose the right queue system (e.g. Kafka, RabbitMQ, SQS).
- Set queue limits and alerting.



Cost vs Latency Trade-offs

- More instances reduce latency but increase cost.
- Batching improves GPU utilization but can increase latency.
- Use autoscaling to match demand.
- Choose right instance types (CPU, GPU, spot instances).
- Consider model optimization (quantization, pruning).
- Find the right balance for your use case.



Common Failure Points



- High latency due to overload.
- GPU or CPU saturation.
- Model loading delays (cold starts).
- Queue build-up and dropped requests.
- Inefficient autoscaling configuration.
- High cost due to over-provisioning.
- Out of memory (OOM) errors.



Best Practices

- Monitor CPU, GPU, memory, latency and queue length.
- Use horizontal scaling and autoscaling.
- Implement batching where possible.
- Optimize model loading and keep instances warm.
- Set appropriate concurrency limits.
- Use queues and backpressure to handle spikes.
- Test different instance types and tune for cost and performance.
- Regularly review metrics and adjust scaling policies.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



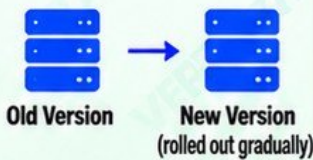
@veriqta



Safe Model Deployment

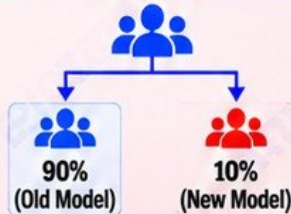
Rolling, canary, shadow and blue-green deployment. Traffic splitting, evaluation signals, automated rollback, model compatibility, and blast-radius control.

Rolling Deployment



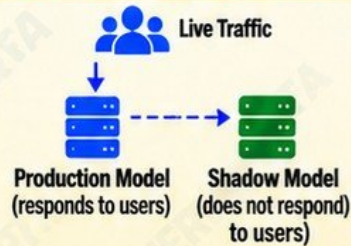
- Replace instances incrementally.
- No downtime.
- Lower risk deployment.
- Useful for small changes or model updates.

Canary Deployment



- Send a small percentage of traffic to the new model.
- Monitor metrics and quality.
- Increase traffic gradually if healthy.
- Quick rollback if issues.

Shadow Deployment



- Send a copy of live traffic to the new model.
- Compare results with production.
- No impact on users.
- Great for evaluation before full rollout.

Blue-Green Deployment



- Run two identical environments (blue and green).
- Switch all traffic to the new version instantly.
- Easy and fast rollback.
- Useful for major model or infrastructure changes.

Traffic Splitting Strategies

- Percentage based**
Send a percentage of traffic to the new model.
- User based**
Route specific users or segments.
- Feature flag based**
Enable for specific features.
- Geographic based**
Roll out by region or location.
- Request based**
Route based on request type or characteristics.

Evaluation Signals

- Model quality metrics** (accuracy, precision, recall)
- Latency and throughput**
- Error rate and success rate**
- Business metrics** (engagement, conversion, etc.)
- Compare predictions** (shadow vs production)
- Monitor data drift and unexpected behaviour**

Automated Rollback

- Define clear rollback conditions (e.g. high error rate, latency increase, quality degradation).
- Use monitoring and alerting.
- Automate rollback to previous model version.
- Keep rollback process tested and reliable.
- Minimize user impact and downtime.

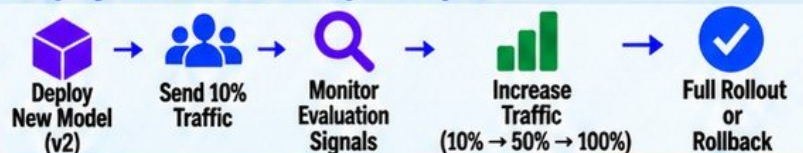
Model Compatibility

- Ensure input and output schema compatibility.
- Validate dependencies and runtime requirements.
- Maintain backward compatibility where possible.
- Test with real data and edge cases.

Blast-Radius Control

- Limit the impact of a bad deployment.
- Use traffic partitioning (by user, region, service).
- Set resource quotas and isolation.
- Monitor closely during rollout.
- Have a clear rollback plan.

Deployment Flow (Canary Example)



Common Failure Points

- Performance degradation (higher latency or errors).
- Poor model quality (inaccurate predictions).
- Incompatibility with production data.
- Resource exhaustion.
- Inadequate monitoring and alerting.
- No clear rollback strategy.

Best Practices

- Start with small traffic (canary) and gradually increase.
- Use shadow deployment for evaluation.
- Monitor both technical and business metrics.
- Automate rollback based on defined thresholds.
- Test model compatibility and dependencies.
- Limit blast radius with traffic segmentation and isolation.
- Keep a well-tested rollback plan.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



ML Observability

Infrastructure + service + model signals. Latency, throughput, errors, saturation, GPU utilization, prediction distribution, feature health and model quality.

What is ML Observability?



- End-to-end visibility into ML systems in production.
- Combines infrastructure, service and model signals.
- Helps detect issues early, maintain reliability and ensure model quality.
- Essential for performance, cost control and trust.

Layers of Observability



Infrastructure Signals
(CPU, GPU, memory, storage, network, nodes)



Service Signals
(latency, throughput, errors, saturation, availability)



Model Signals
(prediction distribution, feature health, model quality)

Key Metrics to Monitor



Latency
Time to get a prediction



Throughput
Requests per second



Errors
Error rates and types



Saturation
CPU, GPU, memory and queue usage



GPU Utilization
GPU usage, memory and temperature



Prediction Distribution
Output values and class balance

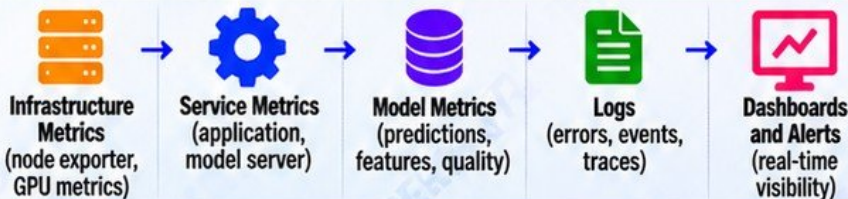


Feature Health
Missing values, schema and data drift



Model Quality
Accuracy, precision, recall and business metrics

ML Observability Stack (Example)



Infrastructure Signals



- CPU utilization
- GPU utilization (e.g. nvidia-smi)
- Memory usage
- Disk and storage I/O
- Network latency and bandwidth
- Node health and availability
- Pod and container status

Service Signals



- Request latency (p50, p95, p99)
- Throughput (requests per second)
- Error rate (4xx, 5xx)
- Queue length
- Concurrency
- Model loading time
- Cold starts
- Service availability and uptime

Model Signals



- Prediction distribution (e.g. histogram, class balance)
- Feature distribution and drift
- Data quality and missing values
- Model accuracy, precision, recall
- Business metrics (conversion, fraud, etc.)
- Output anomalies (unexpected values)
- Model performance by segment (e.g. region, user type)

Feature Health Monitoring



- Check for missing or null values
- Validate feature schema and types
- Monitor feature distribution over time
- Detect data drift and outliers
- Ensure real-time and batch features are consistent
- Alert on broken or stale features

Prediction Distribution (Example)



GPU Monitoring (Example)



- GPU utilization (%)
- GPU memory usage
- GPU temperature
- Power usage
- Processes using GPU
- Detect idle GPUs
- Alert on high or low utilization

Common Issues



- High latency and timeouts
- Increasing error rates
- GPU or CPU saturation
- Data or feature drift
- Prediction distribution changes
- Model quality degradation
- Silent failures (wrong but valid predictions)
- Cost spikes due to inefficient resource use

Best Practices



- Monitor infrastructure, service and model signals
- Set meaningful alerts and thresholds
- Use dashboards for real-time visibility
- Track model quality over time
- Detect data and concept drift early
- Monitor GPU utilization and cost
- Set up automated alerts and incident response
- Regularly review metrics and retrain when needed



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Drift & Model Degradation

Data drift, concept drift, feature drift and performance degradation. Detection signals, false alarms, retraining decisions, and why drift ≠ automatic retraining.

Types of Drift



Data Drift

- Change in input data distribution over time.
- Same problem, different data.
- e.g. new user behaviour, seasonality, new regions.



Feature Drift

- Change in a specific feature distribution.
- Some features behave differently than before.
- e.g. age, location, device type.

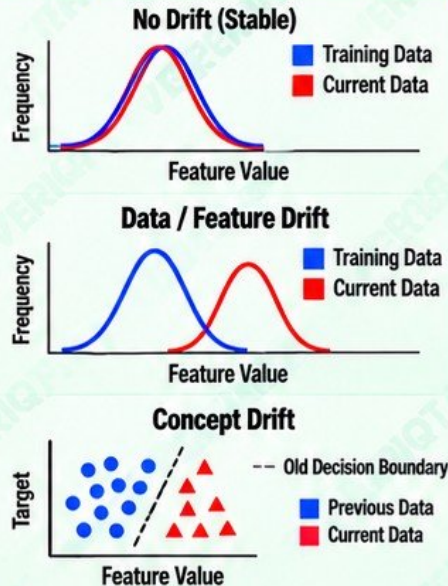


Concept Drift

- Change in the relationship between inputs and outputs.
- Same input now has a different meaning.
- e.g. users behave differently, rules or environment changed.



Examples



Detection Signals



Data Distribution Change

- Statistical tests (e.g. PSI, KS test)
- Monitor feature distributions



Prediction Distribution Change

- Output values and class balance
- Detect unexpected shifts



Model Performance Drop

- Lower accuracy, precision, recall
- Business metric degradation (e.g. conversion, fraud rate)



Online vs Offline Metrics Gap

- Training metrics look good but real world performance drops.



Data Quality Issues

- Missing values, null spikes
- Schema changes, outliers

Why Drift Happens



Changing user behaviour (new trends, seasons)



System or product changes (new features, new user base)



External factors (economy, events, regulations)



Data quality issues (missing data, pipeline changes)



Natural evolution (the world is not static)

Drift vs Performance Degradation

Aspect	Drift	Performance Degradation
What it is	Change in data or relationships	Decrease in model quality or business metrics
Always leads to lower performance?	No	Yes
Examples	Feature distribution change, concept drift	Lower accuracy, higher error rate
Detection method	Statistical tests, distribution checks	Model metrics, business metrics
Action required	Investigate and decide	Mitigate (may require retraining)

Why Drift ≠ Automatic Retraining



- Not all drift is harmful.
- Drift can be temporary (e.g. campaigns, seasonality).
- Retraining is expensive (compute, time, validation).
- May introduce instability or worse performance.
- Need to understand the root cause.
- Must validate that retraining actually improves results.

Retraining Decision Factors



- Is the drift real or a false alarm?
- Is model performance affected?
- What is the business impact?
- Do we have enough new data?
- Has the root cause been identified?
- Will retraining improve results?
- Have we validated the new model?

False Alarms



- Short term spikes or seasonal changes.
- Small statistically significant changes that have no business impact.
- Noisy or unstable metrics.
- Monitoring sensitive to minor variations.
- Misconfigured thresholds.
- Outliers or data pipeline issues.

Best Practices



- Monitor data, model and business metrics continuously.
- Use multiple signals (not just one metric).
- Set meaningful thresholds to reduce false alarms.
- Investigate the root cause of drift.
- Test retraining with proper validation.
- Use staging or shadow deployment.
- Keep historical data and track drift trends.
- Retrain only when there is clear business value.

Common Failure Points



- Ignoring drift until performance drops.
- Retraining too frequently without validation.
- Reacting to false alarms.
- Using only one metric for detection.
- No clear retraining process.
- Overlooking data quality issues.
- Not considering business impact.

Key Takeaways



- Drift is expected in real world systems.
- Not all drift leads to degradation.
- Use data, model and business signals together.
- Investigate before retraining.
- Retraining should be a decision, not an automatic action.
- Well designed monitoring reduces false alarms and keeps models reliable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Reliability Engineering for ML

SLOs for inference, dependency failures, timeouts, retries, fallbacks, graceful degradation, capacity, multi-zone resilience and failure containment.

What is Reliability for ML?



- Ensures ML systems remain available, performant and correct in production.
- Handles failures gracefully without breaking user experience.
- Reduces risk and impact of infrastructure, dependency or model issues.
- Combines traditional reliability practices with ML specific considerations.

SLOs for Inference



Latency SLO
e.g. 95% of requests < 300ms



Availability SLO
e.g. 99.9% successful responses

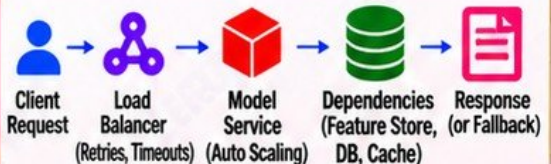


Quality SLO
e.g. minimum accuracy or business metric



User Experience SLO
e.g. low error rate and stable performance

Inference Request Flow (with Reliability Controls)



Reliability Controls



- Timeouts and retries
- Circuit breakers
- Health checks
- Fallback responses
- Graceful degradation
- Monitoring and alerting

Dependency Failures



Database unavailable

- Use retries with backoff
- Fallback to cached data



Feature store failure

- Use timeout and fallback features
- Return default or cached features



External API failure

- Use circuit breaker
- Graceful degradation (simpler model)



Storage failure (model or data)

- Use multi-zone storage
- Keep local cache or fallback mode)

Timeouts, Retries and Circuit Breakers



Timeouts

- Set sensible timeouts (e.g. 15 seconds)
- Avoid long hanging requests



Retries

- Retry on transient failures
- Use exponential backoff
- Avoid retry storms



Circuit Breakers

- Stop sending requests when failure rate is high
- Allow recovery after a cool down period

Fallback and Graceful Degradation



Fallback Model

- Use a simpler or smaller model
- Use last known good model version



Fallback Response

- Return a default or cached response
- Show partial results



Graceful Degradation

- Reduce non essential features
- Keep core functionality available
- Maintain good user experience

Capacity and Scaling



- Plan for traffic spikes and growth
- Use horizontal scaling (more pods)
- Scale based on metrics (CPU, GPU, latency, queue length)
- Keep headroom for unexpected load
- Test scaling under real load

Multi-Zone Resilience



- Deploy across multiple availability zones
- Use global or regional load balancing
- Replicate models and data
- Handle zone failures automatically
- Test failover regularly

Failure Containment



- Isolate failures to a single component or zone
- Use namespaces and resource limits
- Prevent cascading failures
- Use queueing to absorb sudden traffic
- Implement bulkheads (separate resources for critical components)
- Use feature flags for risky changes
- Keep blast radius small

Common Failure Scenarios



- High latency due to dependency slowness
- Model service crashes or pod evictions
- Feature store or database unavailable
- Traffic spike causing resource exhaustion
- Bad model version causing poor results
- Network or zone failure
- Retry storms and cascading failures

Best Practices



- Define clear SLOs and error budgets
- Monitor latency, errors and throughput
- Use health checks and automated recovery
- Implement retries, timeouts and circuit breakers
- Provide fallback and graceful degradation
- Design for multi-zone resilience
- Test failure scenarios regularly
- Keep services and dependencies loosely coupled

Key Takeaways



- Reliability is essential for production ML systems.
- Failures will happen, plan for them.
- Use SLOs, monitoring and automated recovery.
- Graceful degradation keeps users satisfied.
- Design for resilience, scale and failure containment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Security & ML Supply Chain

IAM, workload identity, secrets, artifact integrity, model provenance, container security, dependency risk, dataset access, registry controls and audit trails.

Why Security Matters for ML?



- Protects data, models and infrastructure.
- Prevents unauthorized access and misuse.
- Reduces risk of supply chain attacks.
- Ensures model integrity and trust.
- Required for compliance and auditability.

IAM and Workload Identity



- Use least privilege access.
- Use roles and service accounts (for Kubernetes/cloud).
- Use workload identity instead of long-lived credentials.
- Separate permissions for data, training, inference and registry access.
- Regularly review and rotate access.

Secrets Management



- Store secrets in a secure secrets manager (e.g. cloud secret manager, Vault, Kubernetes secrets).
- Avoid hardcoding secrets in code or images.
- Use short-lived tokens where possible.
- Restrict access using IAM and audit logs.

ML Supply Chain Overview



Artifact Integrity and Provenance



- Verify checksums and signatures.
- Track model provenance (how, when, with what data).
- Use signed artifacts (e.g. Sigstore).
- Store metadata (version, training code, dataset, dependencies).
- Detect tampering or unauthorized changes.

Container Security



- Use minimal base images (small and secure).
- Scan images for vulnerabilities (e.g. Trivy, Gype).
- Keep dependencies updated.
- Run as non-root user.
- Use image signing and policy enforcement.

Dependency Risk



- Monitor open-source dependencies for vulnerabilities.
- Use software bill of materials (SBOM).
- Automatically check for known CVEs.
- Pin versions and avoid untrusted packages.
- Use automated dependency scanning in CI/CD.

Dataset Access and Protection



- Use least privilege access to datasets.
- Protect sensitive and personal data.
- Encrypt data at rest and in transit.
- Track who accessed the data and when.
- Use data masking or anonymization where required.
- Prevent unauthorized data exfiltration.

Registry Controls



- Use private registries for model artifacts and images.
- Restrict who can push and pull.
- Enable image scanning and vulnerability checks.
- Use role-based access control (RBAC).
- Keep audit logs of all registry activities.

Audit Trails and Monitoring



- Log access to data, models, registries and deployments.
- Monitor for unusual activity (e.g. large downloads, unknown access).
- Track changes to models and images.
- Keep audit logs for compliance.
- Integrate with centralized logging and SIEM tools.

Common Threats



- Unauthorized data access.
- Model stealing and tampering.
- Malicious or vulnerable dependencies.
- Insecure container images.
- Compromised credentials.
- Poisoned or manipulated datasets.
- Supply chain attacks.
- Lack of auditing and monitoring.

Best Practices



- Apply least privilege and zero trust principles.
- Use workload identity and short-lived credentials.
- Scan images and dependencies regularly.
- Verify artifact integrity and track provenance.
- Protect datasets and control access.
- Enable comprehensive audit logging.
- Regularly review security policies and access.

Key Takeaways



- Security is essential throughout the ML supply chain.
- Protect data, code, models, containers and infrastructure.
- Use IAM, secrets management and least privilege access.
- Verify artifact integrity and track model provenance.
- Scan images and dependencies for vulnerabilities.
- Control access to datasets and registries.
- Enable audit trails and monitoring for detection and compliance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Cost & Capacity Engineering

CPU vs GPU economics, utilization, batching, right-sizing, autoscaling, spot/preemptible capacity, storage, network transfer and cost-per-inference.

CPU vs GPU Economics



- Lower cost per instance
- Good for smaller models and CPU friendly workloads
- Suitable for pre and post-processing, APIs and lightweight inference



- Higher cost per instance
- Best for large models and high throughput
- Higher performance when utilization is high
- Cost effective when you keep GPUs busy

Utilization



- Track CPU and GPU utilization (CPU %, GPU %, memory %).
- Identify idle or low utilization resources.
- Improve utilization with batching, right-sizing and autoscaling.
- Monitor utilization over time to find patterns.

Batching for Lower Cost



- Process multiple requests together.
- Increases GPU utilization.
- Improves throughput.
- Reduces cost per inference.
- Use dynamic batching where possible.

Right-Sizing



- Choose the right instance type and GPU size for the model.
- Match resources to actual load and performance needs.
- Avoid over-provisioning.
- Regularly review and adjust instance sizes.

Autoscaling



- Scale based on real metrics (e.g. request rate, queue length, GPU utilization).
- Scale out to handle traffic spikes.
- Scale in to reduce cost when idle.
- Use separate scaling rules for different components (e.g. API, model workers).

Spot / Preemptible Capacity



- Use spot or preemptible instances for non critical workloads (e.g. batch inference, training).
- Significantly lower cost compared to on demand.
- Handle interruptions with checkpointing and automatic recovery.
- Keep critical real time workloads on on demand instances.

Storage Costs



- Choose the right storage type (e.g. object storage for datasets, fast storage for model files).
- Use lifecycle policies to move old data to cheaper storage.
- Remove unused model versions and datasets.
- Consider storage cost for logs, checkpoints and artifacts.

Network Transfer Costs



- Minimize data transfer between regions.
- Keep data close to where compute runs.
- Use caching to reduce repeated downloads.
- Be aware of egress costs when serving globally.
- Compress data where possible.

Cost per Inference



- Total cost per inference = (Compute cost + Storage cost + Network costs) ÷ Number of inferences.
- Track this metric regularly.
- Optimize model size, batching and utilization to reduce cost per inference.

Cost Optimization Strategies



- Increase GPU utilization.
- Use batching and async processing.
- Right-size instances.
- Use spot/preemptible capacity.
- Remove idle resources.
- Optimize storage and network usage.
- Use autoscaling.
- Regularly review costs and usage patterns.

Common Cost Pitfalls



- Low GPU utilization (expensive idle capacity).
- Over-provisioning instances.
- Unnecessary data transfer.
- Storing large amounts of unused data.
- Keeping old model versions.
- Not using autoscaling.
- Ignoring monitoring and alerts.

Best Practices



- Design for high utilization.
- Use batching and right-sizing.
- Leverage spot instances for non critical jobs.
- Monitor cost per inference.
- Remove unused resources and data.
- Set up cost alerts and budgets.
- Regularly review and optimize.

Key Takeaways



- GPU resources are expensive, utilization is key.
- Batching, right-sizing and autoscaling reduce cost.
- Spot and preemptible capacity can significantly lower cost.
- Storage and network transfer also impact total cost.
- Monitor cost per inference and optimize regularly.
- Cost optimization does not mean lower performance, it means better resource usage.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



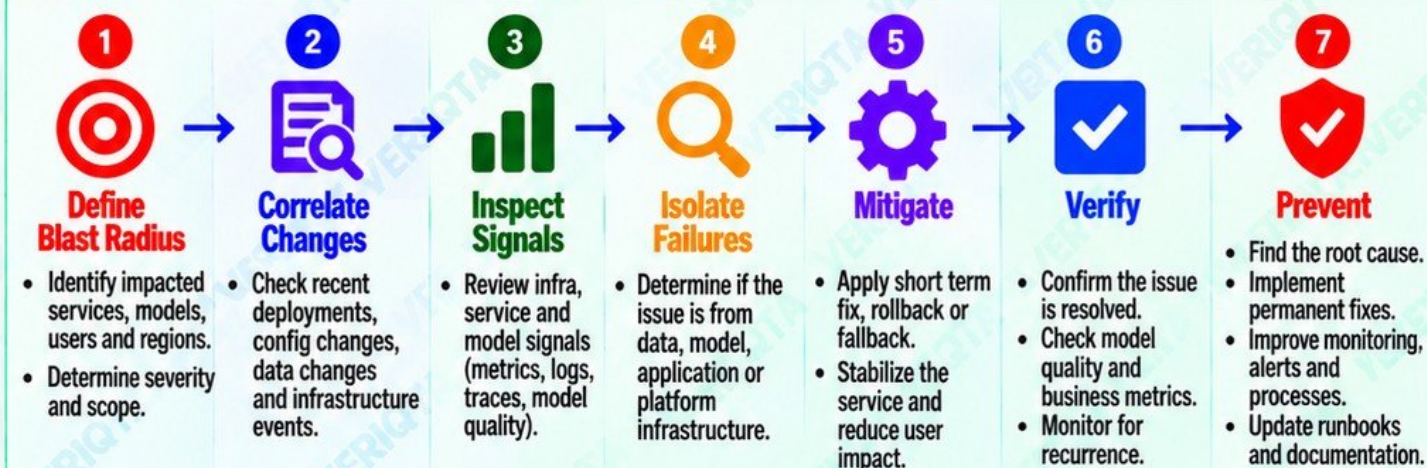
@veriqta



Debugging ML in Production

A senior incident workflow: define blast radius → correlate changes → inspect infra/service/model signals → isolate data/model/platform failures → mitigate → verify → prevent

1. Senior Incident Workflow (End-to-End)



2. What to Inspect (Signals)

- Infrastructure Signals**
 - CPU, GPU, memory, disk, network
 - Node and pod health
 - Autoscaling and capacity events
- Service Signals**
 - Request rate, latency, errors
 - Queue length and timeouts
 - Dependency health (DB, cache, APIs)
- Model Signals**
 - Prediction distribution
 - Model latency and throughput
 - Model errors and fallback rate
- Data Signals**
 - Data freshness and volume
 - Feature distribution and missing values
 - Data pipeline and upstream dependencies

3. Isolating the Root Cause

- Data Issues**
 - Data drift or bad data
 - Missing or incorrect features
 - Delayed or failed data pipeline
- Model Issues**
 - Model degradation
 - Wrong model version
 - Poor prediction quality
 - Model loading or inference errors
- Application/Service Issues**
 - Code bugs
 - Configuration errors
 - Timeouts and retries
 - Dependency failures
- Platform/Infrastructure Issues**
 - Insufficient CPU/GPU
 - Node, pod or cluster issues
 - Network, storage or DNS problems
 - Autoscaling misconfiguration

4. Mitigation Strategies

- Rollback**
 - Revert to a known good model or previous version.
- Fallback**
 - Use a simpler model or rule based system temporarily.
- Scale**
 - Increase replicas or GPU/CPU capacity.
 - Scale queues and workers.
- Configuration Fix**
 - Fix environment variables, secrets, timeouts or resource limits.
- Data Fix**
 - Restore correct data, fix pipeline issues and reprocess if needed.
- Feature Flags**
 - Disable problematic features or routes.

5. Common Issues and Symptoms

- Sudden increase in latency.
- High error rate or timeouts.
- Drop in model quality or unusual predictions.
- GPU or CPU saturation.
- Model not loading or crashes.
- Data pipeline delays or missing data.
- Dependency failures (database, cache, external API).
- Unexpected traffic spike.
- Drift in input data or feature distribution.
- Incorrect model version deployed.

6. Best Practices

- Have clear runbooks for common issues.
- Monitor infrastructure, service and model metrics.
- Set meaningful alerts and thresholds.
- Track model versions and deployments.
- Use canary or shadow deployments.
- Keep fallback options ready.
- Regularly test incident scenarios.
- Document root causes and fixes.
- Continuously improve monitoring and observability.

7. Key Takeaways

- Use a structured incident workflow.
- Always check infra, service, data and model signals.
- Isolate the root cause before applying fixes.
- Mitigate quickly to reduce user impact.
- Verify the fix and monitor closely.
- Prevent recurrence with better monitoring, testing and processes.
- A senior engineer thinks in systems and looks at the full picture.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Production MLOps Platform

Bring everything together: source → data → training → experiments → registry → CI/CD → Kubernetes → serving → observability → feedback. Platform ownership and golden paths.

End-to-End Production MLOps Platform



Key Platform Components

- Source Control**
 - Model code, data pipelines, infrastructure as code
- Data Platform**
 - Ingestion, transformation, feature store, data quality
- Training Platform**
 - Scalable training, distributed training, hyperparameter tuning
- Experiment Tracking**
 - Track runs, parameters, metrics and artifacts
- Model Registry**
 - Version models, metadata, approval and lineage

Deployment and Serving

- CI/CD for ML**
 - Automate build, test, security checks and deploy
- Kubernetes**
 - Run training and inference workloads
- Model Serving**
 - Online, batch and streaming inference
- Traffic Management**
 - Canary, blue green, A/B testing, rollbacks
- Infrastructure**
 - CPU and GPU nodes, autoscaling, spot capacity

Observability and Feedback

- Monitoring**
 - Latency, throughput, errors, resource utilization
- Model Monitoring**
 - Prediction quality, data drift, concept drift, feature health
- Logging and Tracing**
 - Centralized logs, traces and metrics
- Feedback Loop**
 - Collect user feedback, real world data and outcomes
- Continuous Improvement**
 - Retrain, update and optimize models

Platform Ownership

- Set platform standards and golden paths
- Ensure security, compliance and governance
- Manage cost, capacity and reliability
- Provide self service tools for teams
- Enable consistent and repeatable workflows

Golden Paths for Teams

- Predefined, secure and supported workflows
- Templates for common use cases
- Best practices built in (CI/CD, security, monitoring)
- Easy access to training, serving and observability
- Reduces complexity and increases productivity

Benefits

- Faster time to production
- Higher model quality
- Better reliability and observability
- Lower cost and improved efficiency
- Standardized and secure deployments
- Easier collaboration between data, ML and platform teams

Common Challenges

- Siloed tools and teams
- Inconsistent environments
- High infrastructure costs
- Poor model monitoring
- Manual deployments
- Lack of governance and versioning
- Difficulty reproducing results

Best Practices

- Use modular and open source tools
- Implement CI/CD for all ML workflows
- Automate testing and validation
- Monitor both infrastructure and model metrics
- Use Kubernetes for scalability
- Implement cost controls and autoscaling
- Maintain clear documentation and runbooks

Key Takeaways

- Production MLOps brings the full lifecycle together
- A well designed platform reduces risk, cost and complexity
- Golden paths enable teams to move faster and ship reliable models
- Continuous feedback improves model performance over time



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

